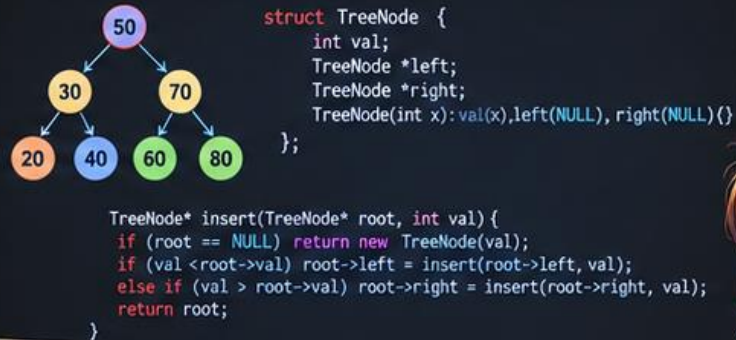




CC1002 - Introducción a la Programación

CLASE 10 : ARBOLES

Departamento de Ciencias de la Computación (DCC) - FCFM
UNIVERSIDAD DE CHILE



```
struct TreeNode {  
    int val;  
    TreeNode *left;  
    TreeNode *right;  
    TreeNode(int x): val(x), left(NULL), right(NULL) {}  
};
```

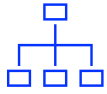
```
TreeNode* insert(TreeNode* root, int val) {  
    if (root == NULL) return new TreeNode(val);  
    if (val < root->val) root->left = insert(root->left, val);  
    else if (val > root->val) root->right = insert(root->right, val);  
    return root;  
}
```

UNIVERSIDAD DE CHILE FCFM

```
struct TreeNode {  
    int val;  
    TreeNode *left;  
    TreeNode *right;  
    TreeNode(int x): val(x), left(NULL), right(NULL) {}  
};
```

```
TreeNode* find(TreeNode* root, int key) {  
    if (root == NULL) return NULL;  
    if (root->val == key) return root;  
    if (key < root->val) return find(root->left, key);  
    return find(root->right, key);  
}
```

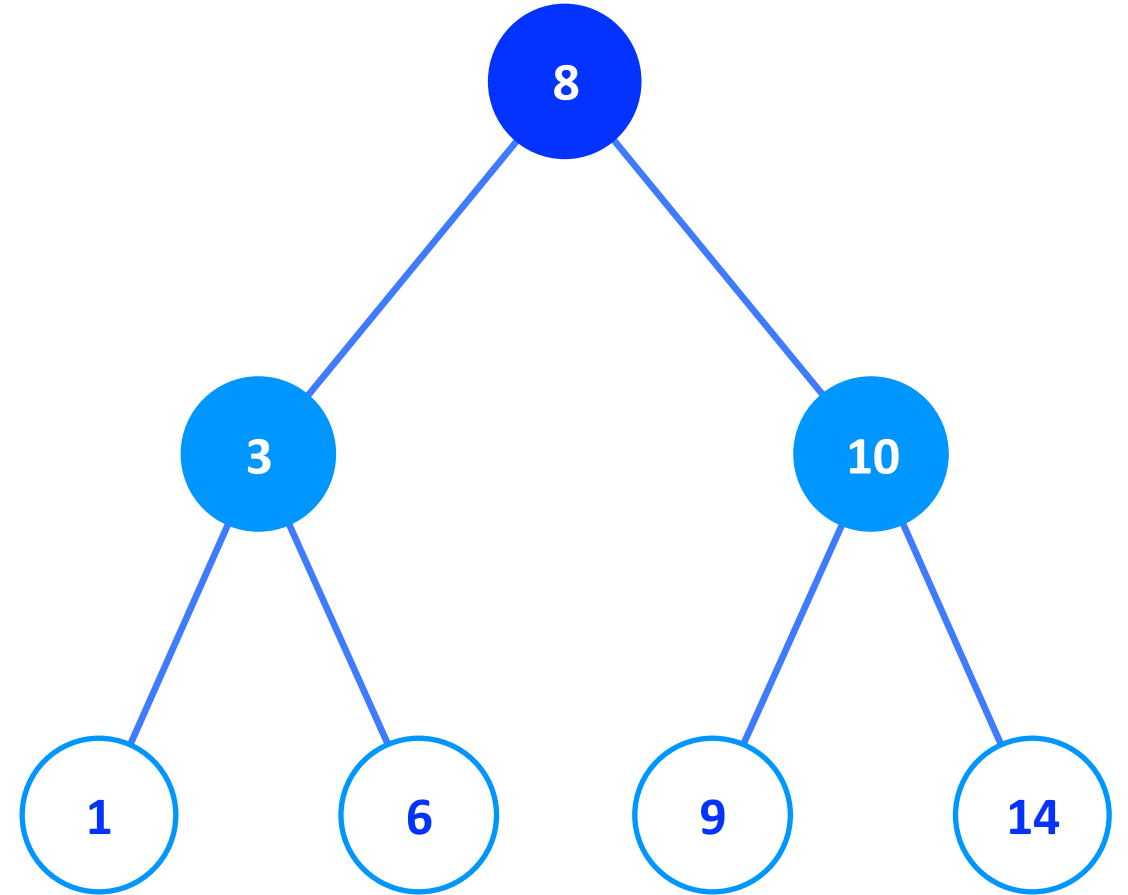
Nadia Decar
Eduardo Godoy



CLASE 10 · ESTRUCTURAS DE DATOS

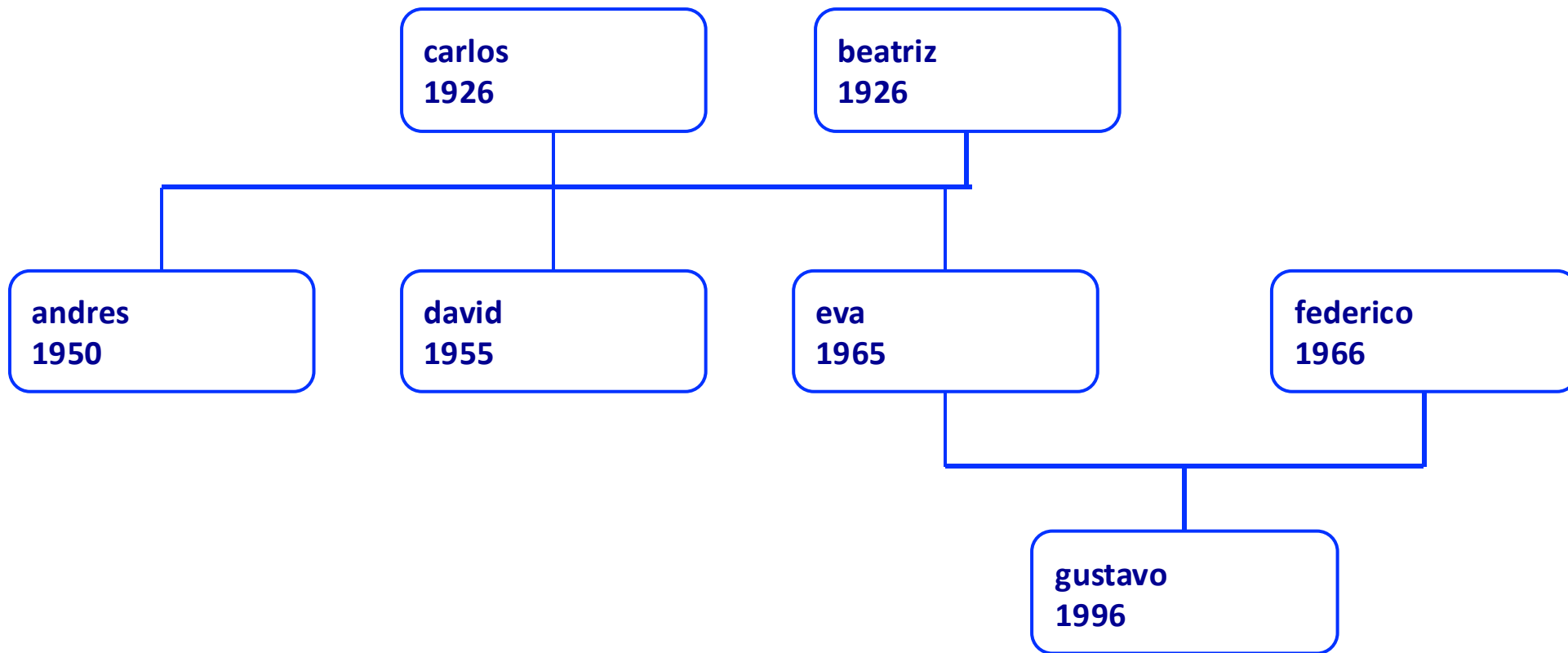
Árboles

Jerarquías, recorridos y árboles binarios de búsqueda



Problema #1

Diseñe una estructura para representar el siguiente árbol genealógico de ancestros



Estructura para representar ancestros

```
import estructura
```

```
# persona: nombre(str) nacimiento(int) padre(persona) madre(persona)  
estructura.crear("persona", "nombre nacimiento padre madre")
```

```
# primera generacion ("abuelos")
```

```
carlos    = persona("carlos", 1926, None, None)  
beatriz   = persona("beatriz", 1926, None, None)
```

```
# segunda generacion ("padres")
```

```
andres    = persona("andres", 1950, carlos, beatriz)  
david     = persona("david", 1955, carlos, beatriz)  
eva       = persona("eva", 1965, carlos, beatriz)  
federico  = persona("federico", 1966, None, None)
```

```
# tercera generacion ("nietos")
```

```
gustavo   = persona("gustavo", 1996, federico, eva)
```

¿Cómo decidimos si una persona es ancestro de otra?

```
# esAncestro: persona persona -> bool
# devuelve True si persona x es ancestro de persona P
# ej: esAncestro(carlos, gustavo) devuelve True
def esAncestro(x, P):
    if P == None:
        return False
    if P.padre == None and P.madre == None:
        return False

    # ancestro por parte de padre?
    p = P.padre != None and (P.padre == x or esAncestro(x, P.padre))

    # ancestro por parte de madre?
    q = P.madre != None and (P.madre == x or esAncestro(x, P.madre))

    return p or q

assert esAncestro(carlos, gustavo)
```

Lista de ancestros de una persona

```
# ancestros: persona -> lista(persona)
# lista con ancestros de persona P
# ej: ancestros(david) es lista(carlos, lista(beatriz, listaVacía))
def ancestros(P):
    if P == None:
        return listaVacía
    # L1: lista de ancestros por parte de padre
    if P.padre == None:
        L1 = listaVacía
    else:
        L1 = lista(P.padre, ancestros(P.padre))
    # L2: lista de ancestros por parte de madre
    if P.madre == None:
        L2 = listaVacía
    else:
        L2 = lista(P.madre, ancestros(P.madre))
    # ancestros por parte de padre y de madre
    return juntar(L1, L2)
```

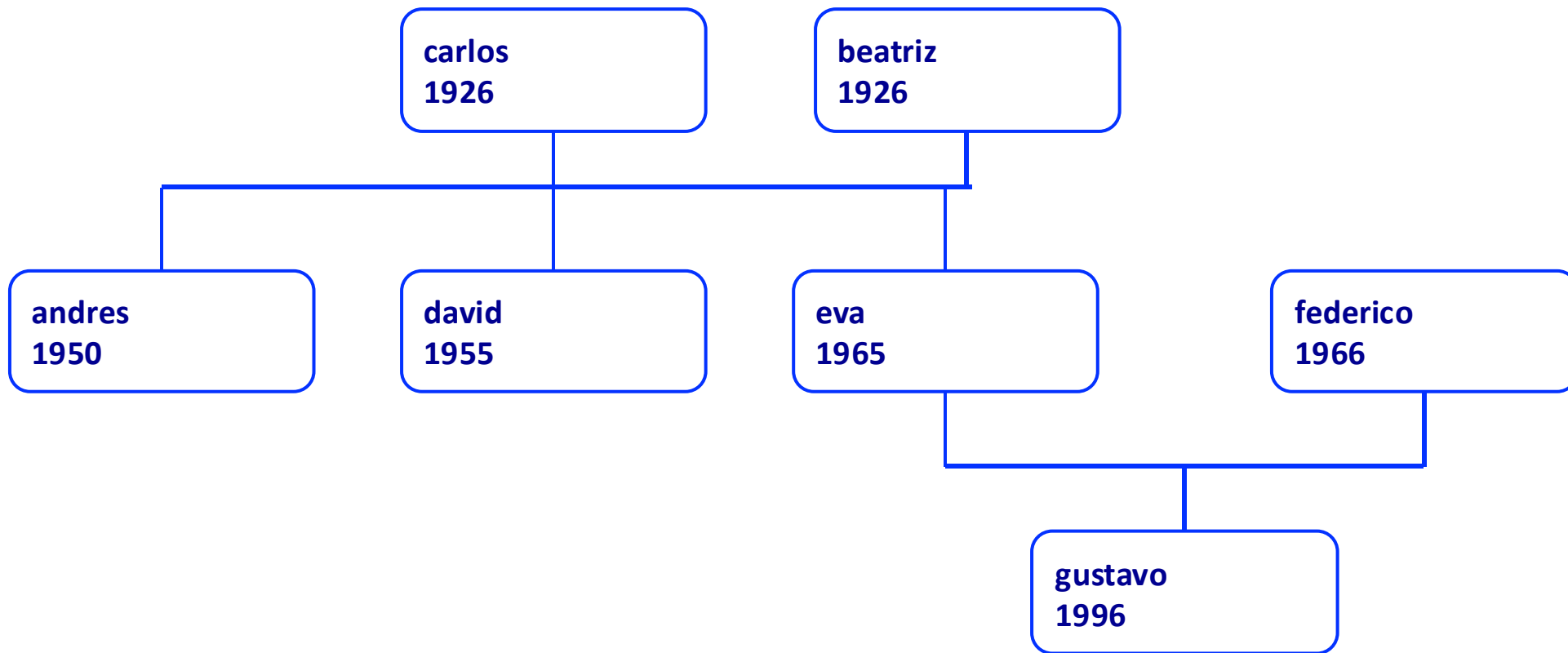
Función auxiliar para unir dos listas

```
# juntar: lista(any) lista(any) -> lista(any)
# une dos listas entregando una nueva sin repeticiones
def juntar(L1, L2):
    assert esLista(L1) and esLista(L2)
    if L2 == listaVacia:
        return L1
    elemento = cabeza(L2)
    if not enLista(elemento, L1):
        L1 = lista(elemento, L1)
    return juntar(L1, cola(L2))

# enLista: any lista(any) -> bool
# devuelve True si elemento esta en lista L
def enLista(e, L):
    assert esLista(L)
    if L == listaVacia:
        return False
    if e == cabeza(L):
        return True
    return enLista(e, cola(L))
```

Problema #2

Diseñe una estructura para representar el siguiente árbol genealógico de descendientes



Estructura para representar descendientes

```
import estructura
```

```
# persona: nombre(str) nacimiento(int) hijos(Lista(persona))  
estructura.crear("persona", "nombre nacimiento hijos")
```

```
# nietos
```

```
gustavo = persona("gustavo", 1996, listaVacia)
```

```
# padres
```

```
andres = persona("andres", 1950, listaVacia)
```

```
david = persona("david", 1955, listaVacia)
```

```
eva = persona("eva", 1965, lista(gustavo, listaVacia))
```

```
federico = persona("federico", 1966, lista(gustavo, listaVacia))
```

```
# abuelos
```

```
hijosCB = lista(andres, lista(david, lista(eva, listaVacia)))
```

```
carlos = persona("carlos", 1926, hijosCB)
```

```
beatriz = persona("beatriz", 1926, hijosCB)
```

¿Cómo decidimos si una persona es descendiente de otra?

```
# esDescendiente: persona persona -> bool  
# devuelve True si persona x es descendiente de P  
# ej: esDescendiente(gustavo, carlos) devuelve True  
def esDescendiente(x, P):  
    if P == None:  
        return False  
    if enLista(x, P.hijos):  
        return True  
    return esDescendienteHijos(x, P.hijos)
```

Funciones mutuamente recursivas

```
# esDescendienteHijos: persona lista(persona) -> bool
# devuelve True si persona x es descendiente de alguna
# persona de la lista L
# ej: esDescendienteHijos(gustavo, hijosCB) es True
def esDescendienteHijos(x, L):
    if L == listaVacia:
        return False
    if esDescendiente(x, cabeza(L)):
        return True
    return esDescendienteHijos(x, cola(L))

# test de funciones mutuamente recursivas
assert esDescendiente(gustavo, carlos)
assert esDescendienteHijos(gustavo, hijosCB)
```

Lista de descendientes de una persona

```
# descendientes: persona -> lista(persona)
# lista de personas descendientes de P
def descendientes(P):
    if P == None or P.hijos == None:
        return listaVacia
    L = P.hijos
    return juntar(L, descendientesLista(L))

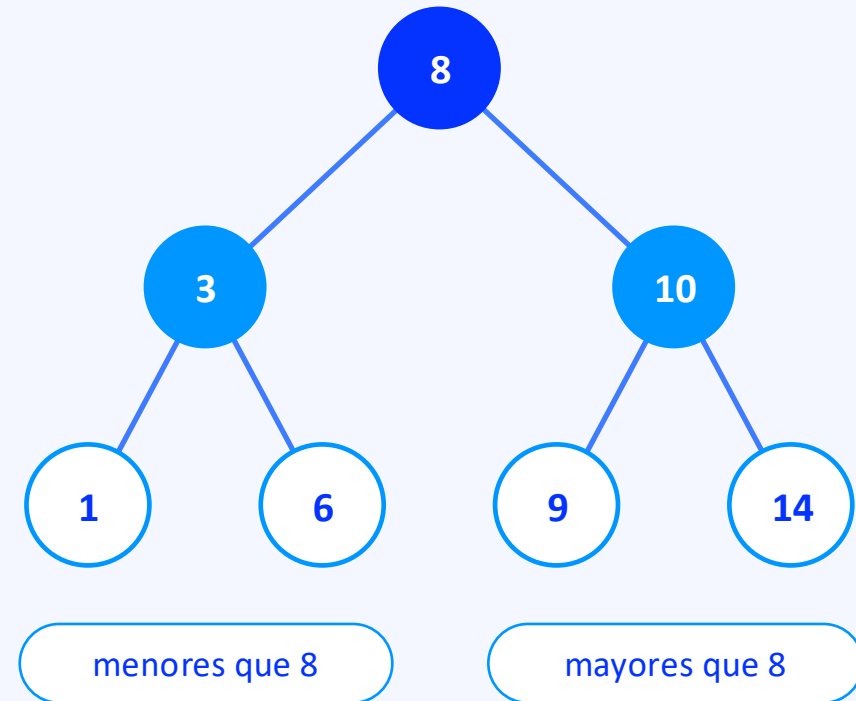
# descendientesLista: lista(persona) -> lista(persona)
# lista de descendientes de personas de lista L
def descendientesLista(L):
    if L == listaVacia:
        return listaVacia
    return juntar(descendientes(cabeza(L)),
                  descendientesLista(cola(L)))
```

Definición

Árbol Binario de Búsqueda (ABB)

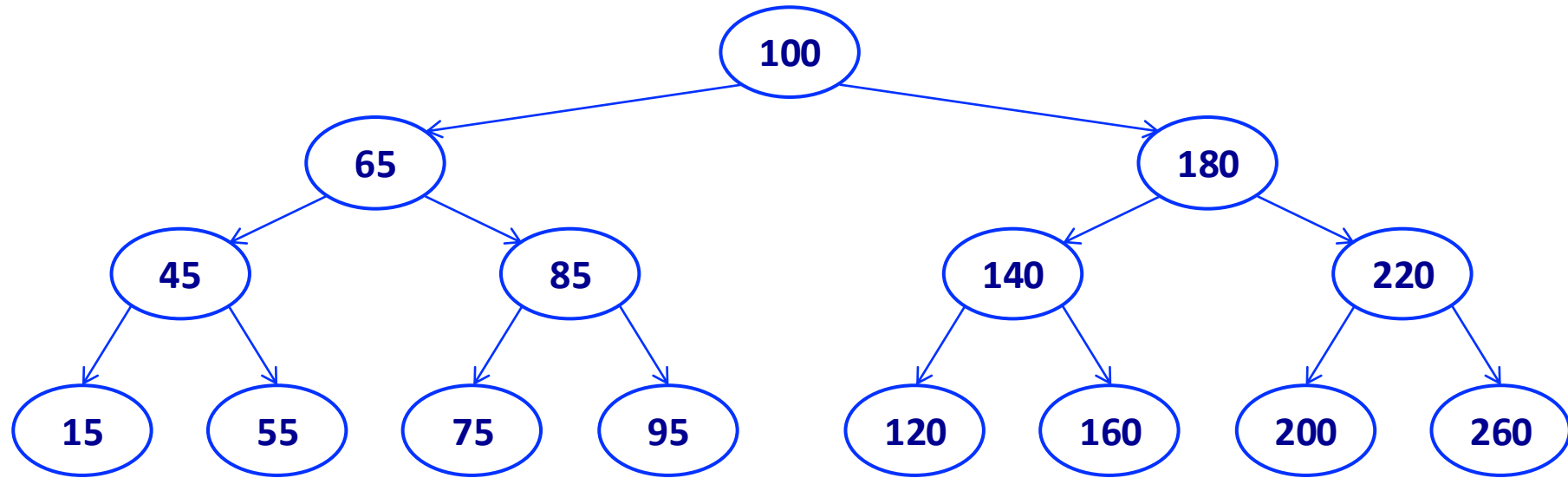
Un ABB es un árbol binario que:

- Es un nodo vacío, o bien
- Es un nodo con dos hijos: izquierdo y derecho
- Sus hijos también son ABB
- Los hijos de la izquierda son menores que la raíz
- Los hijos de la derecha son mayores que la raíz

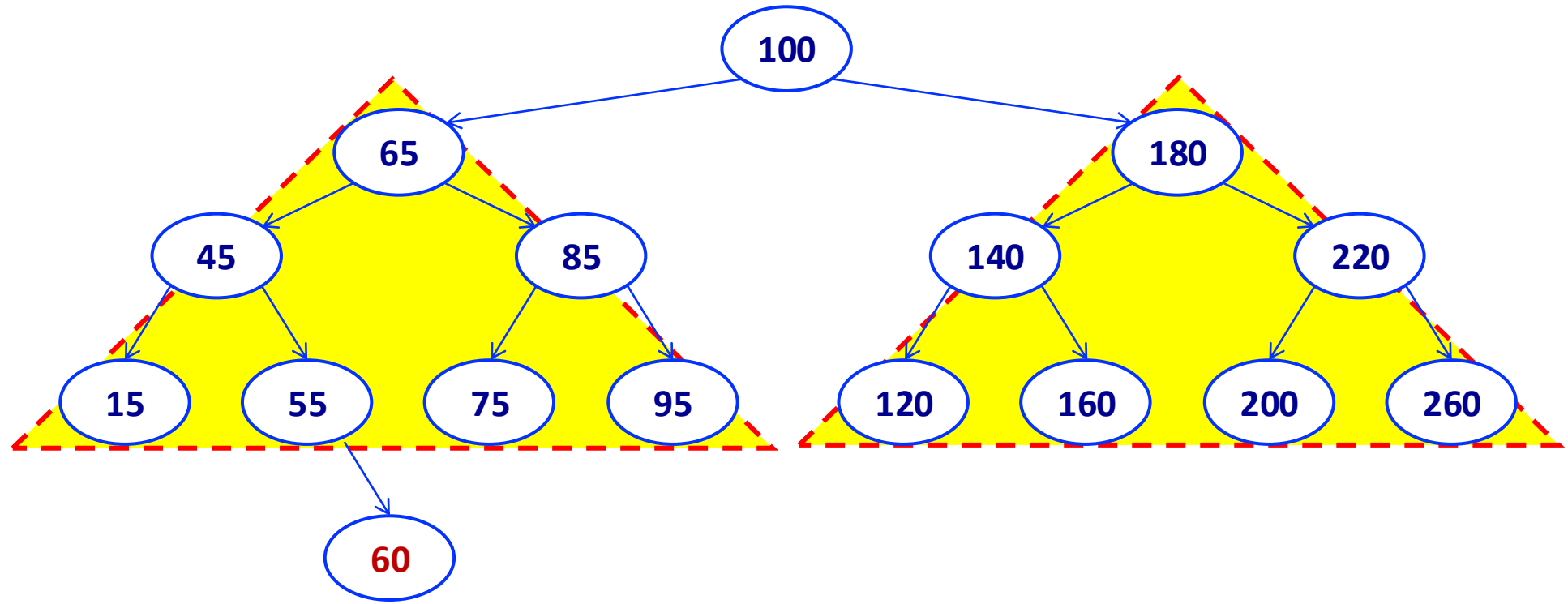


Ejemplo de ABB con raíz 8

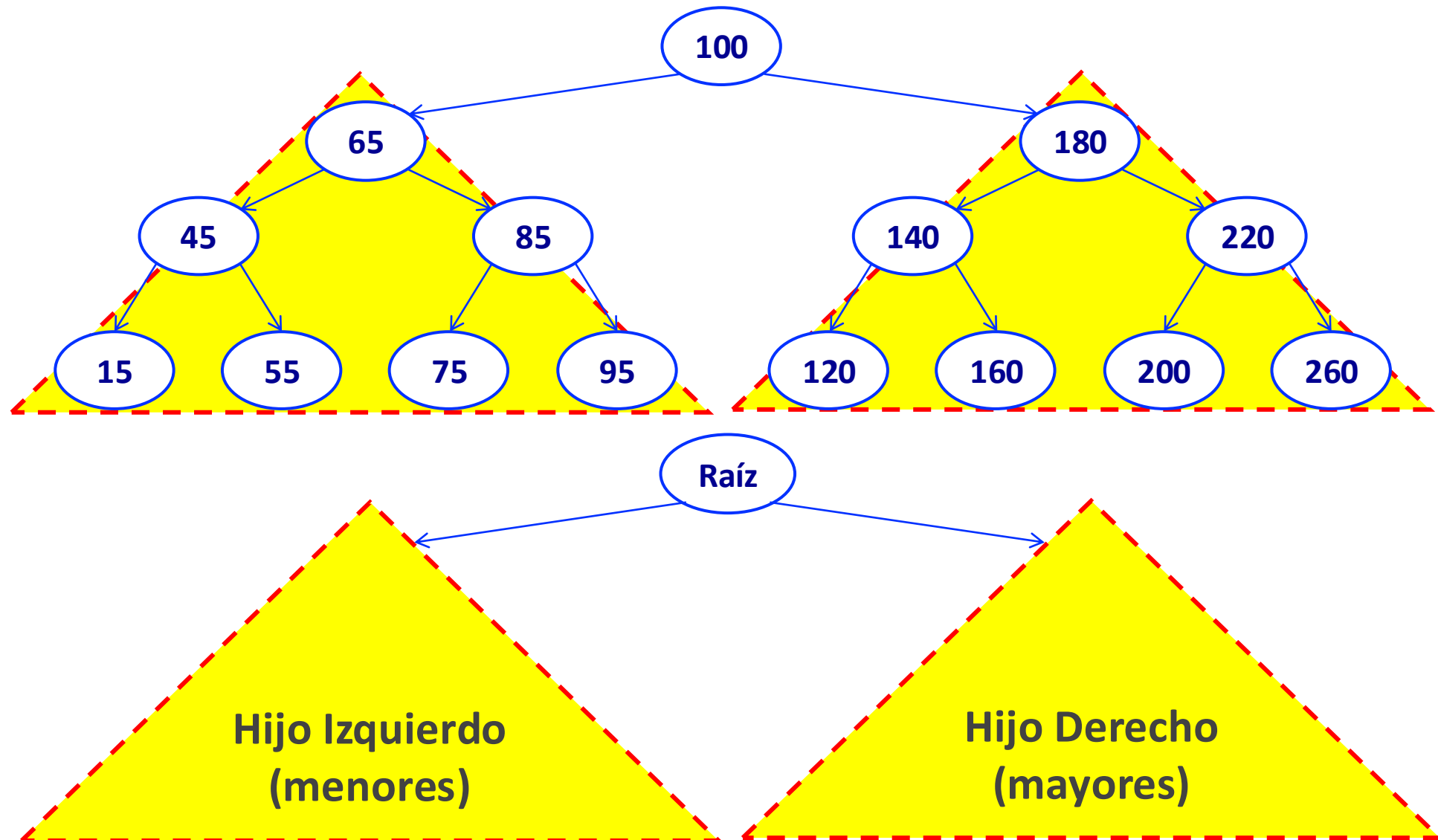
Ejemplo (visual)



Ejemplo (visual)



Ejemplo (visual)



Propiedades

- Al partir, un ABB está sin contenido:
`def crearABB( ):`
 `return nodoVacio`
- Los hijos de la izquierda son menores que la raíz
- Los hijos de la derecha son mayores que la raíz
- No hay 2 nodos con igual valor

Uso:
`raiz = crearABB()`

Describiendo un ABB

- Nodo:
 - Contenido del árbol
 - Mantiene el **valor**
 - Mantiene una referencia al hijo **izquierdo** y una referencia al hijo **derecho**
- El **valor** puede ser:
 - Un número
 - Una lista
 - Una estructura
 - ... etc.

Describiendo un ABB

- Nodo:
 - Contenido del árbol
 - Mantiene el **valor**
 - Mantiene una referencia al hijo **izquierdo** y una referencia al hijo **derecho**
- El **valor** puede ser:
 - Un número
 - Una lista
 - Una estructura
 - ... etc.

• Nota: Por lo general es una estructura:

```
estructura.crear("nodo", "valor izq der")
```

crearNodo

- Nota: Por lo general es una estructura:

```
estructura.crear("nodo", "valor izq der")
```

```
# crearNodo: valor nodo nodo -> nodo  
# devuelve un nodo con datos de valor y los  
# nodos izq y der, respectivamente  
def crearNodo(valor, izq, der):  
    return nodo(valor, izq, der)
```

Insertar un valor

- función **insertarABB(valor, raiz)**:
 - permite poner valores en un Árbol Binario de Búsqueda o **ABB**
 - Si valor es **menor** que la raiz, pondrá el valor en el hijo **izquierdo**
 - Si valor es **mayor** que la raiz, pondrá el valor en el hijo **derecho**

Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo
- Ejemplo para un ABB de números

```
raiz = crearABB()
raiz = insertarABB(100, raiz)
raiz = insertarABB(65, raiz)
raiz = insertarABB(180, raiz)
```

Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo
- Ejemplo para un ABB de números

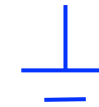
```
raiz = crearABB()
raiz = insertarABB(100, raiz)
raiz = insertarABB(65, raiz)
raiz = insertarABB(180, raiz)
```

El orden de este ABB está
dado por el mismo valor

Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo

- Ejemplo para un ABB de números
raiz = crearABB()

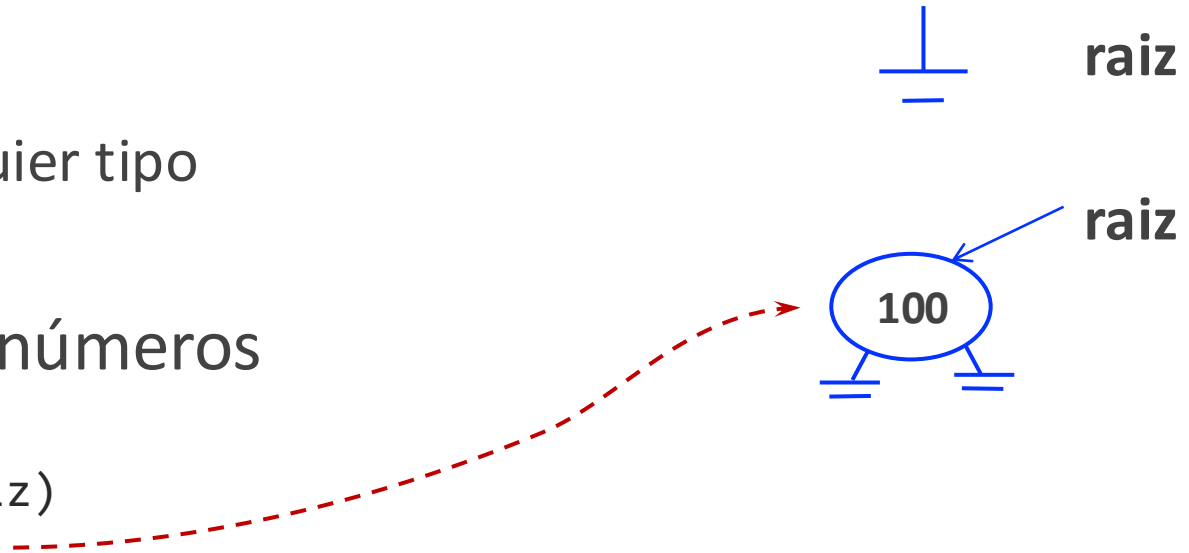


raiz

Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo
- Ejemplo para un ABB de números

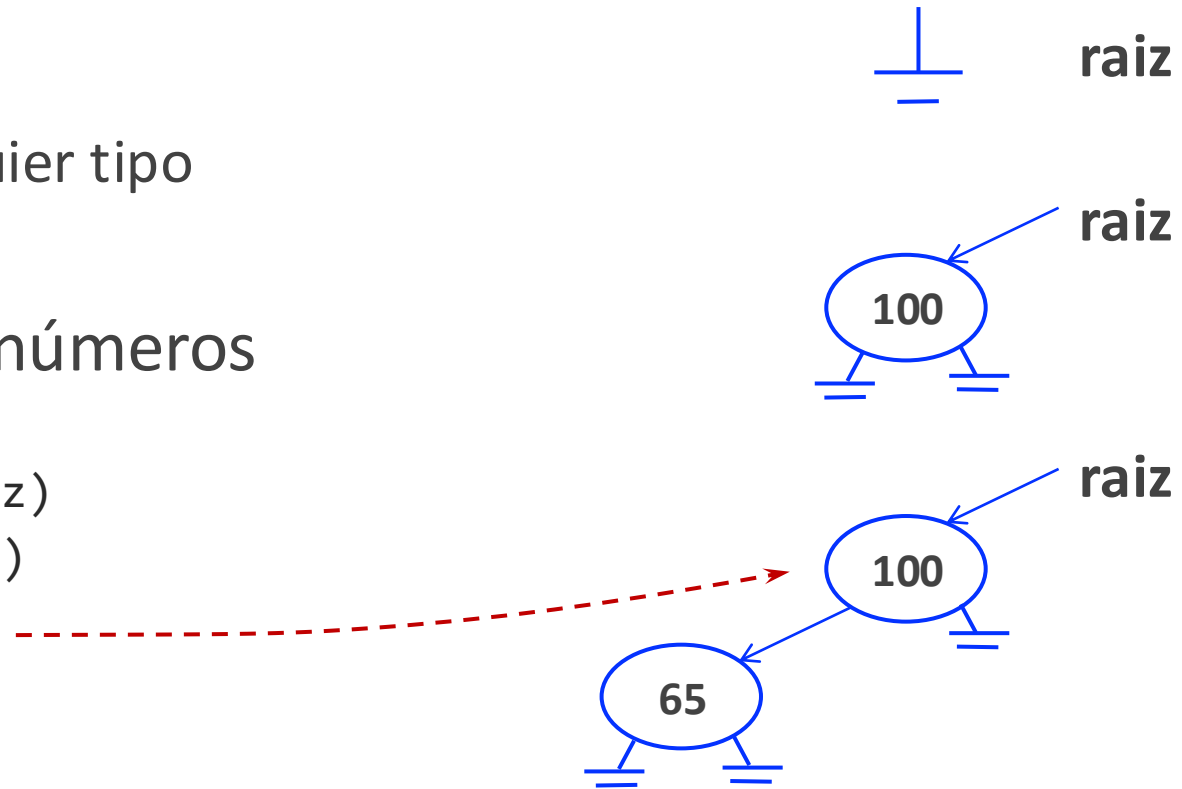
```
raiz = crearABB()  
raiz = insertarABB(100, raiz)
```



Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo
- Ejemplo para un ABB de números

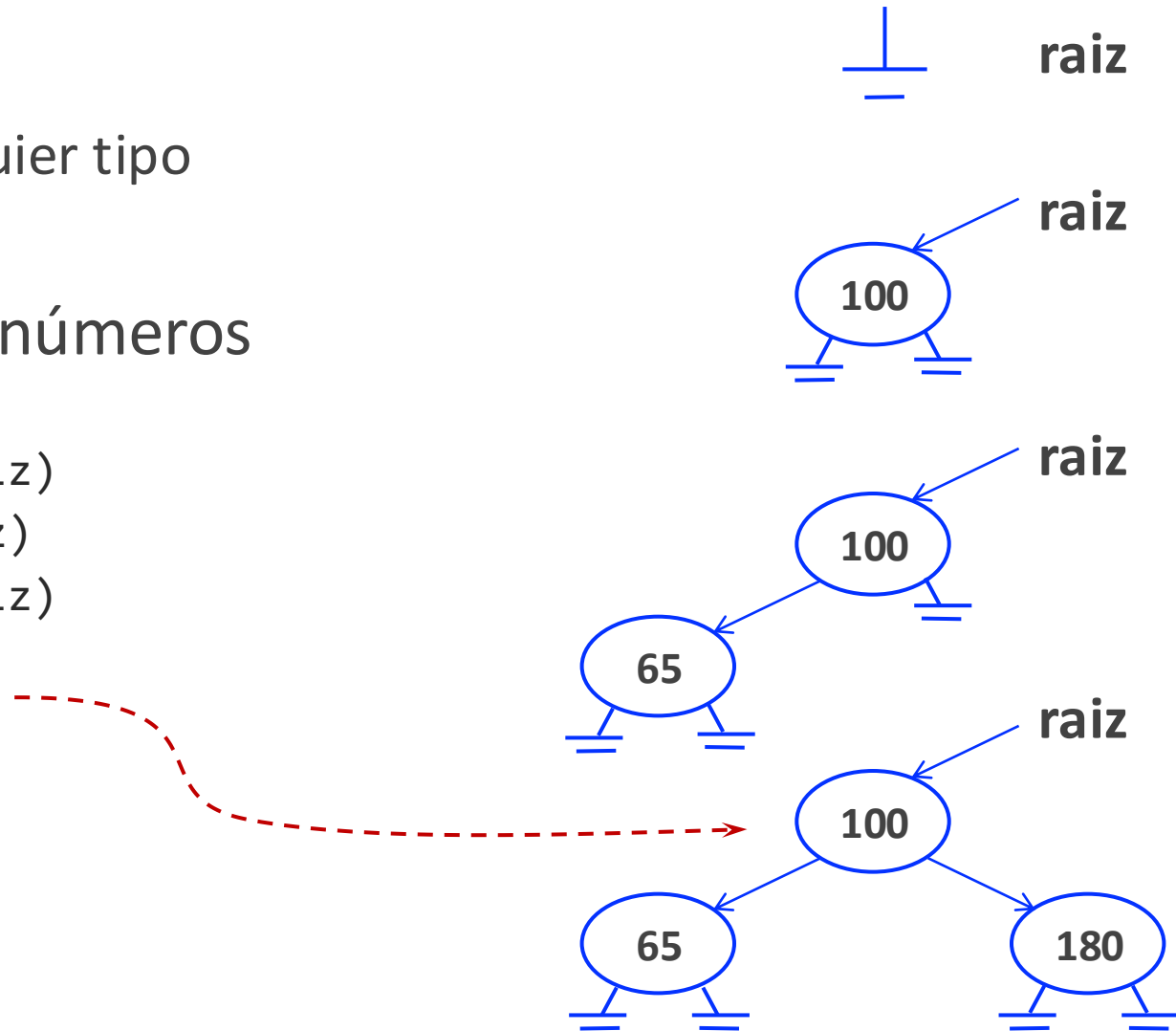
```
raiz = crearABB()
raiz = insertarABB(100, raiz)
raiz = insertarABB(65, raiz)
```



Insertar y valor

- insertarABB(valor, raiz)
 - **valor** puede ser de cualquier tipo
- Ejemplo para un ABB de números

```
raiz = crearABB()
raiz = insertarABB(100, raiz)
raiz = insertarABB(65, raiz)
raiz = insertarABB(180, raiz)
```



Un nodo: una estructura **no** mutable

Insertar y valor

- Ejemplo para un ABB de **personas**:

```
estructura.crear("persona", "nombre edad rut")
juan = persona("juan", 25, "11111111-1")
maria= persona("maria", 33, "77777777-7")
ana= persona("ana", 19, "12345678-9")
raiz = crearABB( )
raiz = insertarABB(juan, raiz)
raiz = insertarABB(maria, raiz)
raiz = insertarABB(ana, raiz)
```

Insertar y valor

- Ejemplo para un ABB de **personas**:

```
estructura.crear("persona", "nombre edad rut")
juan = persona("juan", 25, "11111111-1")
maria= persona("maria", 33, "77777777-7")
ana= persona("ana", 19, "12345678-9")
raiz = crearABB( )
raiz = insertarABB(juan, raiz)
raiz = insertarABB(maria, raiz)
raiz = insertarABB(ana, raiz)
```

**El orden de este ABB
hay que definirlo!**

Insertar y valor

- "persona" se puede ordenar por:
 - Nombre
 - Edad
 - Rut
- Una vez definido el orden NO se cambia

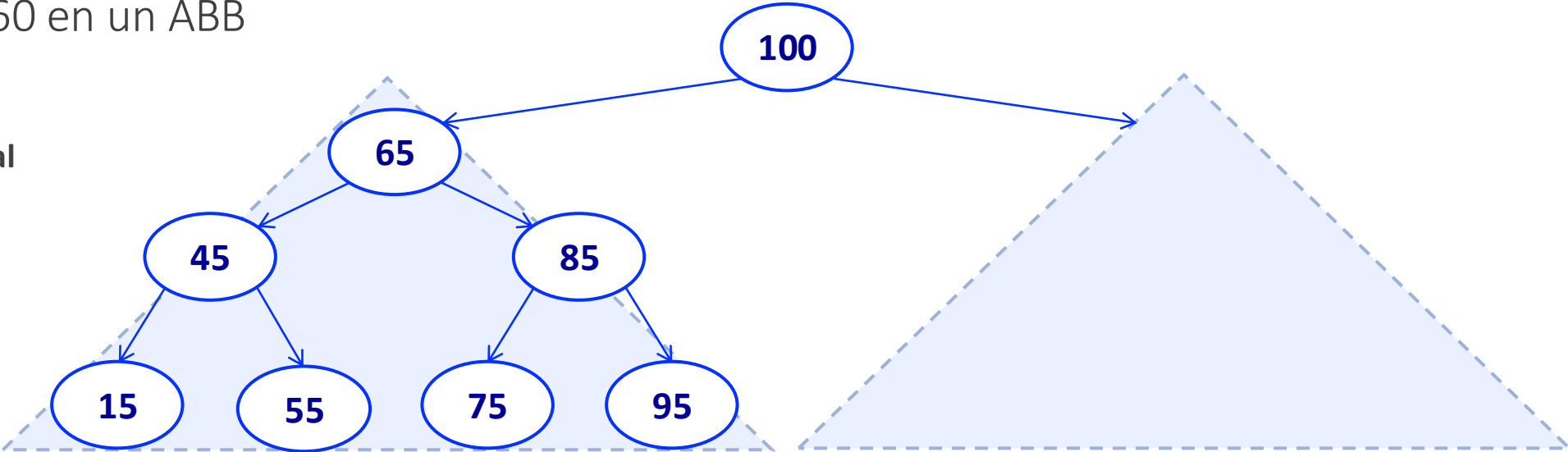
Insertar y valor

- Insertando "persona" usando el rut

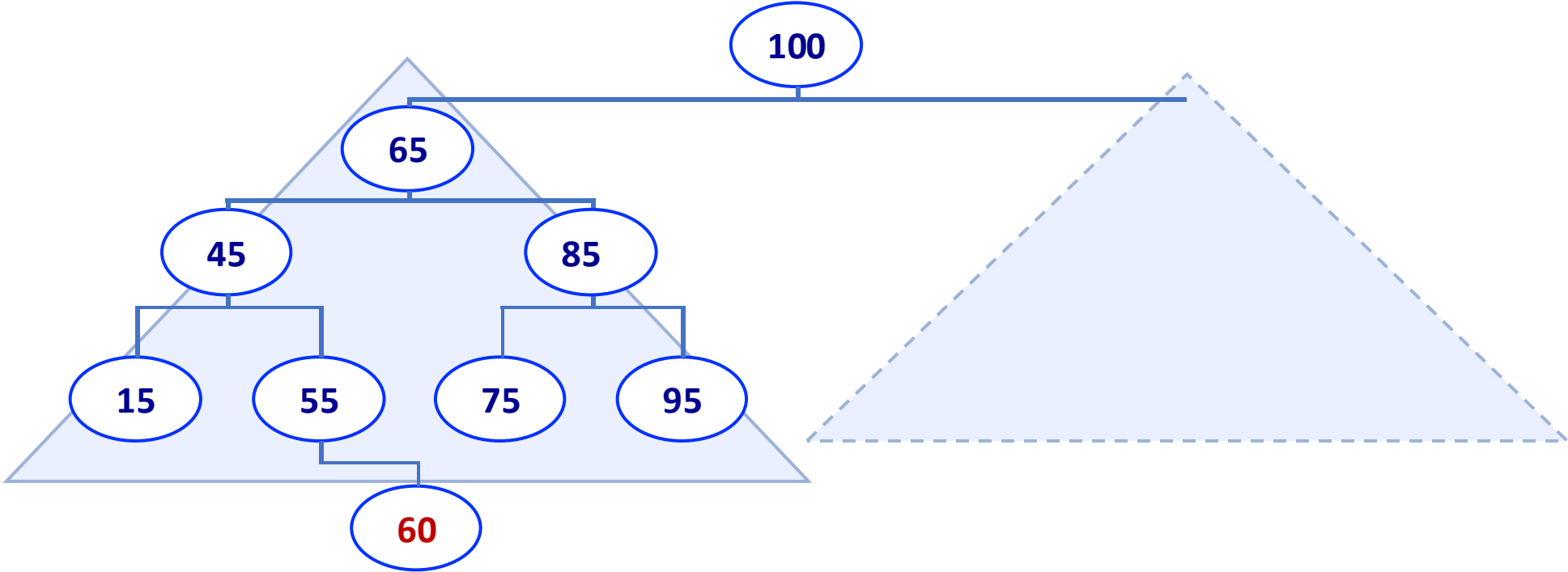
```
# insertarABB: persona nodo -> nodo
# inserta un nodo con atributo valor en el ABB raiz
def insertarABB(valor, raiz):
    if raiz == nodoVacio:
        return crearNodo(valor, nodoVacio, nodoVacio)
    else:
        # Test: no se pueden insertar rut repetido
        assert raiz.valor.rut != valor.rut
        # Insercion
        if valor.rut < raiz.valor.rut:
            return crearNodo(raiz.valor, insertarABB(valor, raiz.izq), raiz.der)
        else: # valor.rut > raiz.rut
            return crearNodo(raiz.valor, raiz.izq, insertarABB(valor, raiz.der))
```

Insertar 60 en un ABB

Estado inicial



Después de insertar 60



Insertar y valor

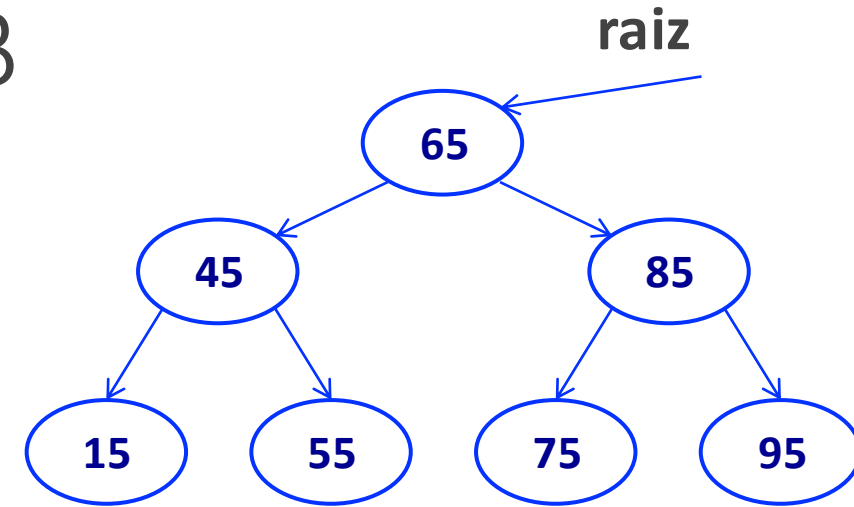
- Insertando "persona" usando el rut

```
# insertarABB: persona nodo -> nodo
# inserta un nodo con atributo valor en el ABB raiz
def insertarABB(valor, raiz):
    if raiz == nodoVacio:
        return crearNodo(valor, nodoVacio, nodoVacio)
    else:
        # Test: no se pueden insertar rut repetido
        assert raiz.valor.rut != valor.rut
        # Insercion
        if valor.rut < raiz.valor.rut:
            return crearNodo(raiz.valor, insertarABB(valor, raiz.izq), raiz.der)
        else: # valor.rut > raiz.rut
            return crearNodo(raiz.valor, raiz.izq, insertarABB(valor, raiz.der))
```

```
# crearNodo: persona nodo nodo -> nodo
# devuelve un nodo con datos de valor y los
# nodos izq y der, respectivamente
def crearNodo(valor, izq, der):
    return nodo(valor, izq, der)
```

Recorriendo un ABB

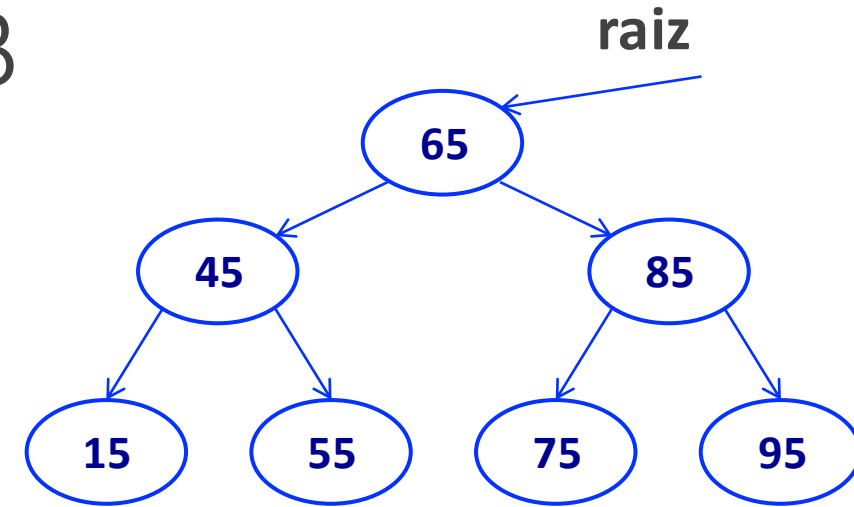
- **Cómo recorrer el contenido del ABB de menor a mayor**



1. Antes de acceder a la raíz (R), primero hay que acceder/recorrer el árbol izquierdo (I)
 2. Finalizado el recorrido del árbol izquierdo (I) se accede al contenido de la raíz (R)
 3. Luego de acceder la raíz, hay que acceder/recorrer el árbol derecho (D)
- Se denota como I-R-D

Recorriendo un ABB

- **Cómo recorrer el contenido del ABB de menor a mayor**



- En el recorrido se puede:
 1. Escribir inmediatamente el contenido
 - `def` escribirContenido_IRD(raiz)
: :
 2. Crear un "string" con todo el contenido
 - `def` contenidoEnString_IRD (raiz)
: :

Escribiendo el contenido de un árbol de personas en pantalla

```
def escribirContenido_IRD(raiz)
    if raiz == nodoVacio:
        print (" ")
    else:
        escribirContenido_IRD(raiz.izq)
        persona = raiz.valor
        print ("Nombre: " + persona.nombre)
        print ("Rut: " + persona.rut)
        print ("Edad: " + persona.edad)
        escribirContenido_IRD(raiz.der)
```

Escribiendo el contenido de un árbol de personas en pantalla

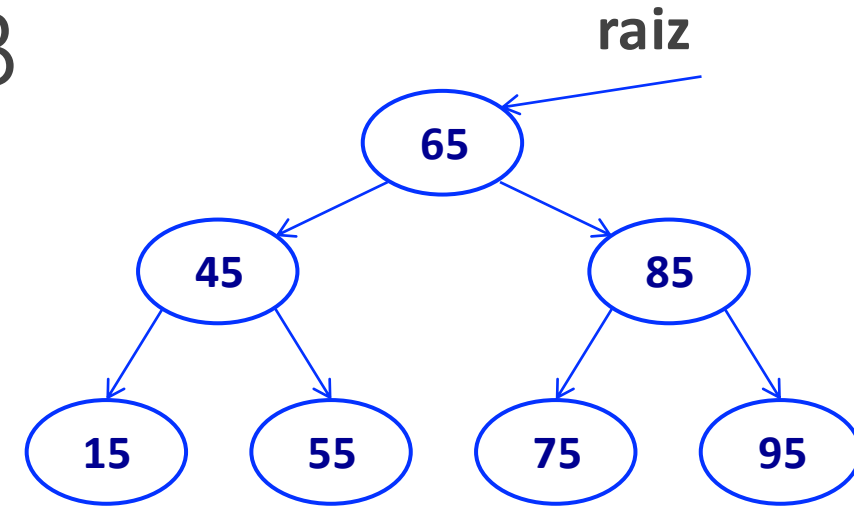
```
def escribirContenido_IRD(raiz)
    if raiz == nodoVacio:
        print (" ")
    else:
        escribirContenido_IRD(raiz.izq)
        persona = raiz.valor
        : :
        : :
        : :
        escribirContenido_IRD(raiz.der)
```

Escribiendo el contenido de un árbol de personas en pantalla

```
def escribirContenido_IRD(raiz)
    if raiz == nodoVacio:
        print (" ")
    else:
        escribirContenido_IRD(raiz.izq)
        persona = raiz.valor
        print ("Nombre: " + persona.nombre)
        print ("Rut: " + persona.rut)
        print ("Edad: " + persona.edad)
        escribirContenido_IRD(raiz.der)
```

Recorriendo un ABB

- **Cómo recorrer el contenido del ABB de menor a mayor**



- En el recorrido se puede:
 1. Escribir inmediatamente el contenido
 - **def** escribirContenido_IRD(raiz)
: :
 2. Crear un "string" con todo el contenido
 - **def** contenidoEnString_IRD (raiz)
: :

Todo el contenido de un árbol de personas en un String

```
def contenidoEnString_IRD(raiz)
  if raiz == nodoVacio:
    return " "
  else:
    persona = raiz.valor
    texto = "Nombre: " + persona.nombre
    texto = texto + " Rut: " + persona.rut
    texto = texto + " Edad: " + str(persona.edad)
    return contenidoEnString_IRD (raiz.izq) + texto +
      contenidoEnString_IRD (raiz.der)
```

Todo el contenido de un árbol de personas en un String

```
def contenidoEnString_IRD(raiz)
  if raiz == nodoVacio:
    return " "
  else:
    persona = raiz.valor
    texto = "Nombre: " + persona.nombre
    texto = texto + " Rut: " + persona.rut
    texto = texto + " Edad: " + str(persona.edad)
    return contenidoEnString_IRD (raiz.izq) + texto +
      contenidoEnString_IRD (raiz.der)
```

Uso:

```
: :
print ("El contenido del arbol es: ")
print (contenidoEnString_IRD (arbolDePersonas))
: :
```

Cuidado!!

Uso:

```
:: :  
print ("El contenido de arbol es: ")  
escribirContenido_IRD(arbolDePersonas)  
:: :
```

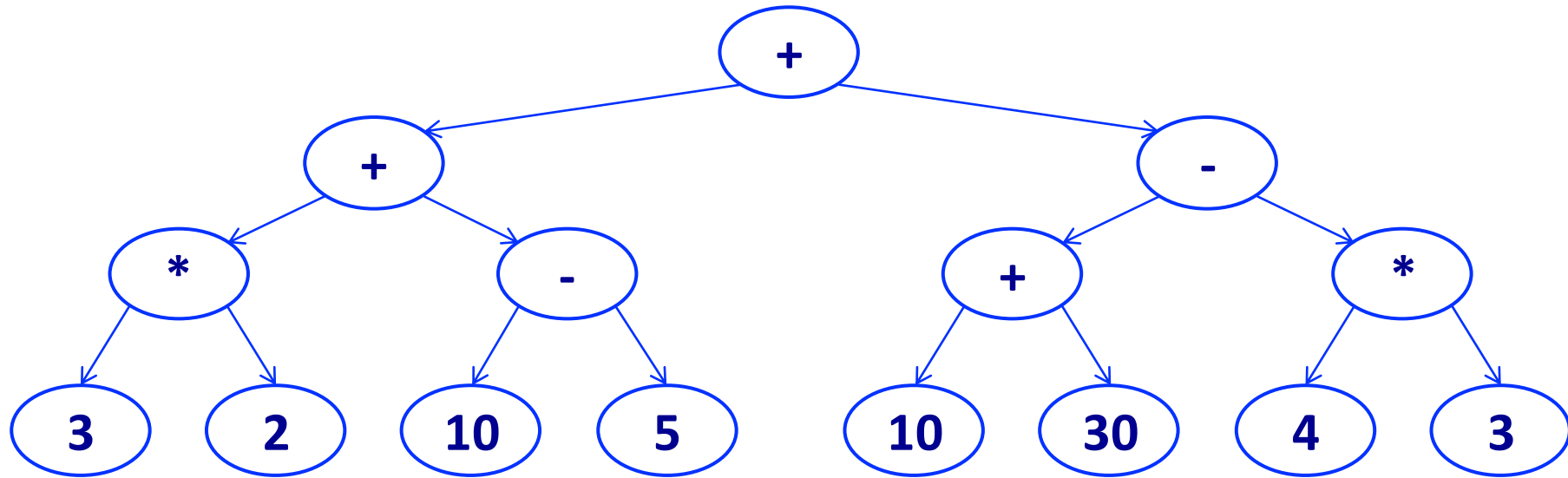
En este caso es la función
la que escribe el contenido
del árbol.

Uso:

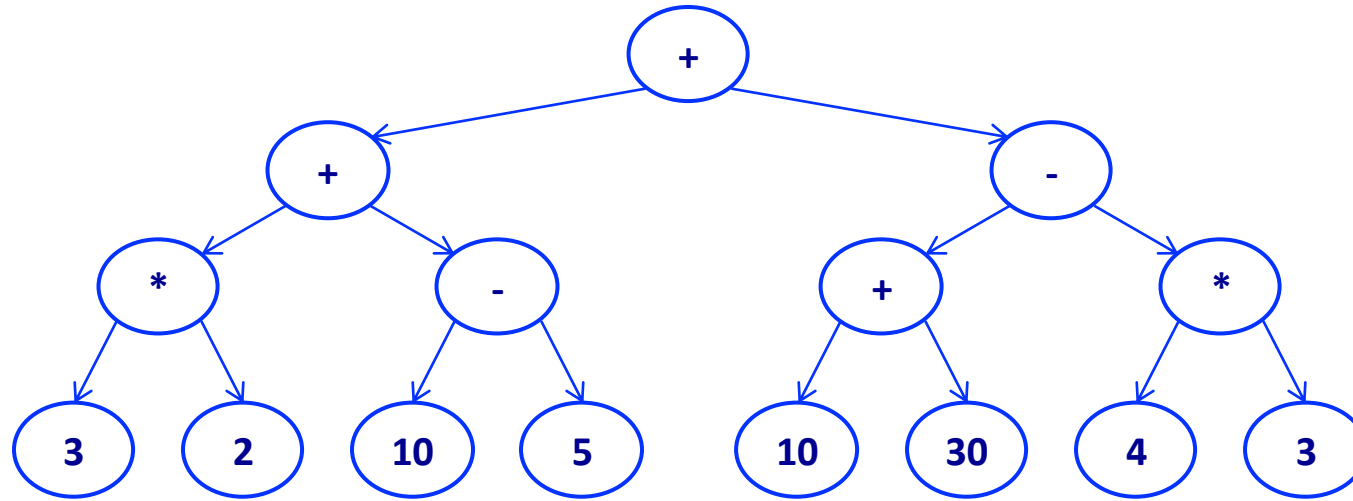
```
:: :  
print("El contenido del arbol es: ")  
print(contenidoEnString_IRD (arbolDePersonas))  
:: :
```

En este caso la función
retorna el contenido del
árbol en un string.
Este contenido se puede
guardar en una variable o
bien escribir en pantalla

Un árbol de expresiones matemáticas



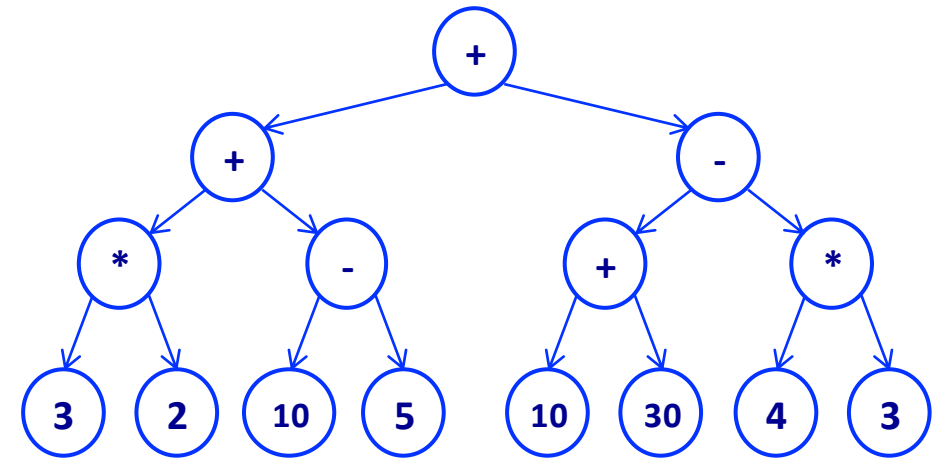
- Se recorre en orden IDR:
 - Lo primero que se evaluará es: $3 * 2 \rightarrow 6$
 - Lo segundo que se evaluará es: $10 - 5 \rightarrow 5$
 - Lo tercero que se evaluará es: $6 + 5 \rightarrow 11$
 - Etc.



- **Árbol simple de expresiones:**
 - Sólo operaciones simples (+, -, *, /)
 - Sólo operaciones binarias (requieren de 2 valores)
 - El árbol no puede estar vacío

Evaluar un árbol de expresiones

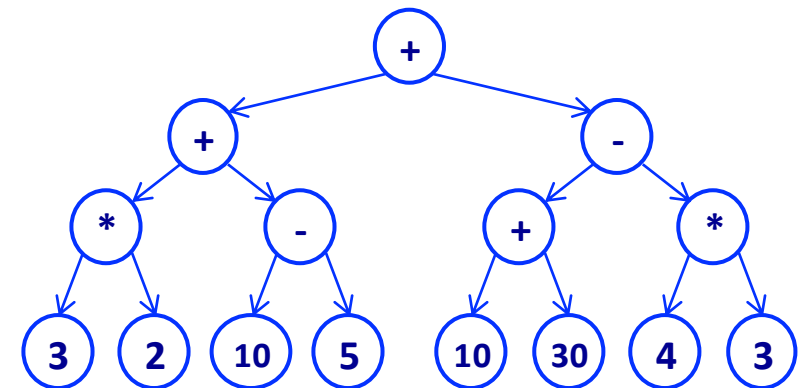
```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```



Evaluar un árbol de expresiones

```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```

La raíz no puede ser nula

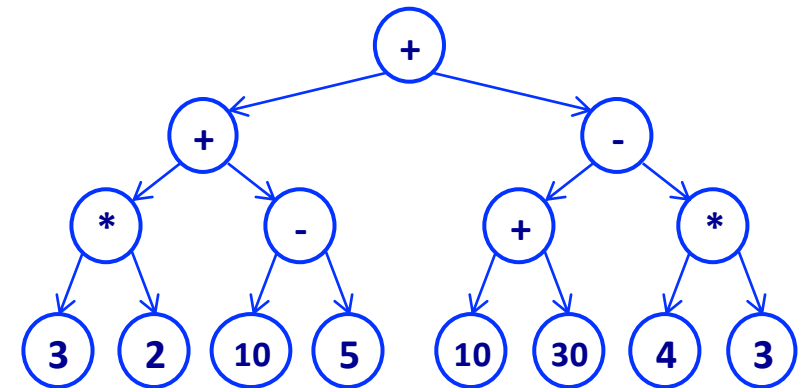


Evaluar un árbol de expresiones

```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```

La raíz no puede ser nula

Si no hay hijos, es una hoja
luego es un número



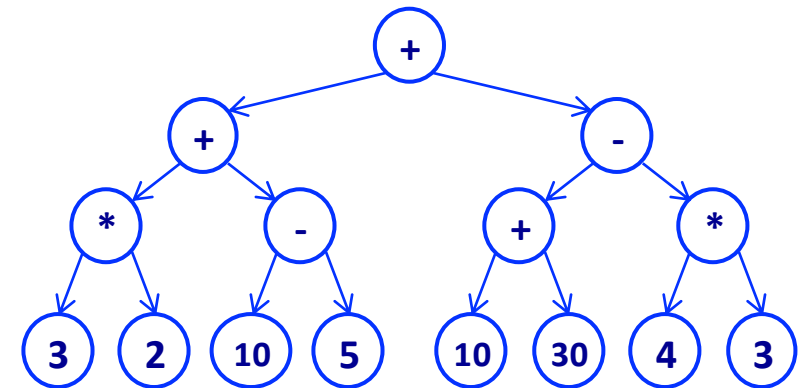
Evaluar un árbol de expresiones

```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```

La raíz no puede ser nula

Si no hay hijos, es una hoja
luego es un número

Es un nodo interno y contiene
un operador



Evaluar un árbol de expresiones

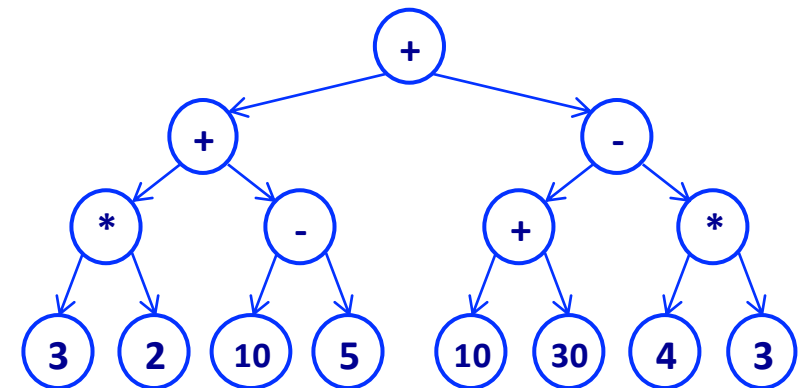
```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```

La raíz no puede ser nula

Si no hay hijos, es una hoja
luego es un número

Es un nodo interno y contiene
un operador

Se recorren los hijos izquierdo
y derecho
El recorrido debe retornar
un valor



Evaluar un árbol de expresiones

```
def evaluar(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return raiz.valor  
    else:  
        valorIzquierdo = evaluar(raiz.izq)  
        valorDerecho = evaluar(raiz.der)  
        operador = raiz.valor  
        if operador == "+":  
            return valorIzquierdo + valorDerecho  
        elif operador == "-":  
            return valorIzquierdo - valorDerecho  
        elif operador == "*":  
            return valorIzquierdo * valorDerecho  
        elif operador == "/":  
            return valorIzquierdo / valorDerecho
```

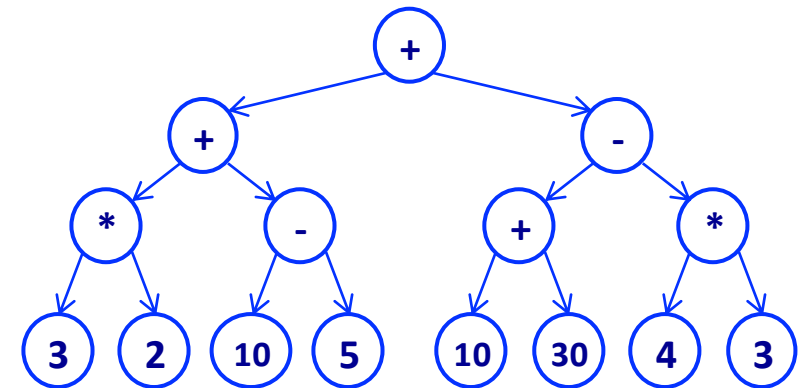
La raíz no puede ser nula

Si no hay hijos, es una hoja
luego es un número

Es un nodo interno y contiene
un operador

Se recorren los hijos izquierdo
y derecho
El recorrido debe retornar
un valor

Se evalúa el operador



¿Cuántos nodo tiene en árbol?

- Solución :
 - Si el árbol está vacío $\rightarrow 0$
 - En caso contrario $\rightarrow 1 + \text{el número de nodos izquierdo} + \text{el número de nodos derechos}$

¿Cuántos nodos tiene un árbol?

```
def contarNodos(raiz):  
    if raiz == nodoVacio:  
        return 0  
    else:  
        return 1 + contarNodos(raiz.izq) + contarNodos(raiz.der)
```

Caso base

Un árbol vacío tiene 0 nodos.

Caso recursivo

1 (la raíz) + los nodos del hijo izquierdo + los del hijo derecho

¿Está balanceado el árbol?

- Un árbol está balanceado si:

1

El árbol vacío está balanceado.

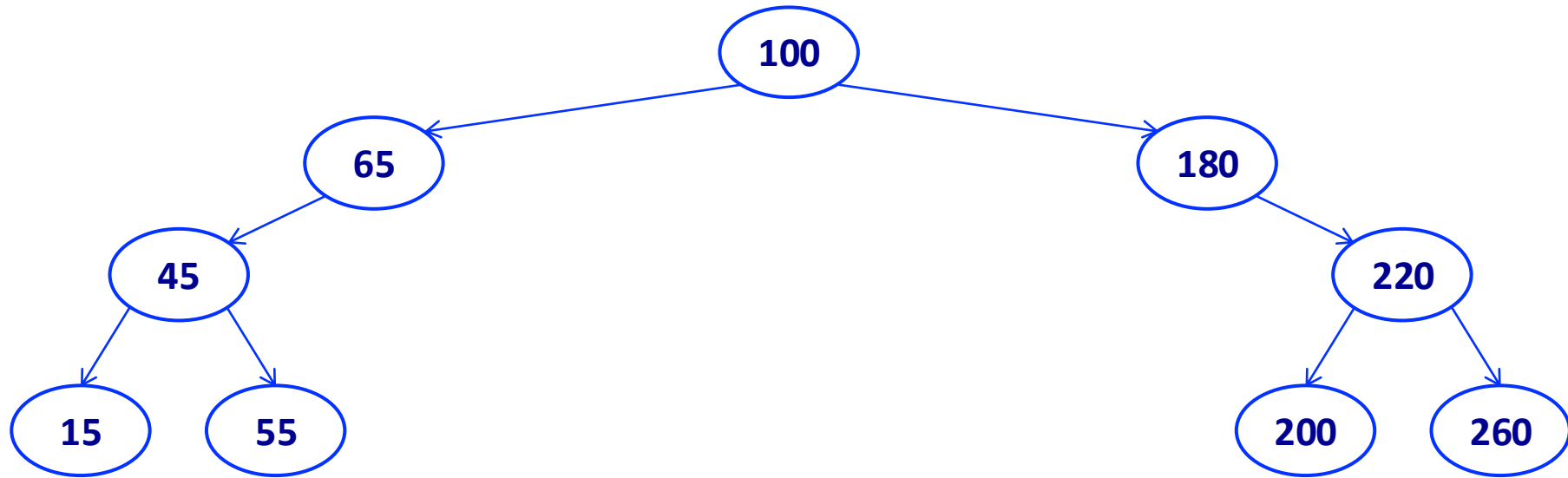
2

La diferencia entre el número de nodos izquierdo y derecho es menor o igual que 1.

3

El árbol izquierdo y el árbol derecho están balanceados.

Ejemplo: Árbol **no** balanceado



¿Por qué no está balanceado?

El nodo 65 tiene 3 nodos a la izquierda (45, 15, 55) y ninguno a la derecha: la diferencia es 3, mayor que 1.

¿Está balanceado el árbol?

```
def balanceado(raiz)
    if raiz == nodoVacio:
        return True
    else:
        total_Izq = contarNodos(raiz.izq)
        total_Der = contarNodos(raiz.der)
        if Math.abs(total_izq - total_der) > 1 :
            return False
        else:
            return balanceado(raiz.izq) and balanceado(raiz.der)
```

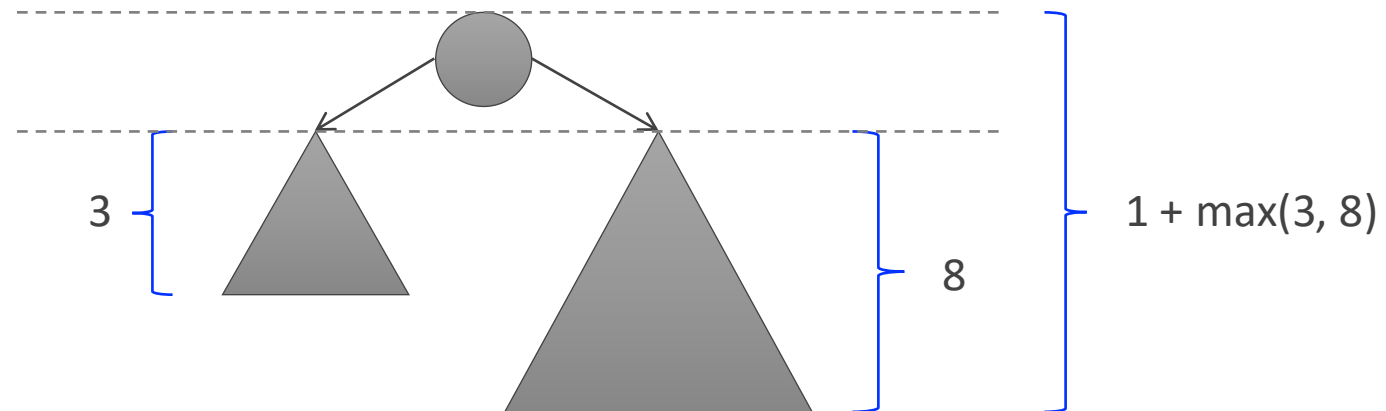
Se apoya en contarNodos para comparar ambos lados y luego se aplica, recursivamente, a los dos hijos.

Propuestos

- Cuál es la profundidad máxima de un árbol binario?
- Escribir el contenido de un ABB de mayor a menor
- Retornar un string con la expresión matemática (sin evaluar) de un árbol de expresiones

Cuál es la profundidad máxima de un árbol binario?

- Estrategia:
 - La profundidad de un árbol vacío es cero
 - Para una nodo no vacío
 - "medir" la profundidad de su árbol izquierdo
 - "medir" la profundidad de su árbol derecho
 - Luego, la nueva profundidad será:
 - 1 más el máximo de las medidas anteriores



Cuál es la profundidad máxima de un árbol binario?

```
def profundidad(raiz):  
    if raiz == nodoVacio:  
        return 0  
    else:  
        prof_izq = profundidad(raiz.izq)  
        prof_der = profundidad(raiz.der)  
        return 1 + max(prof_izq , prof_der)
```

Escribir el contenido de un ABB de mayor a menor

```
def escribirContenido_DRI(raiz)
    if raiz == nodoVacio:
        print (" ")
    else:
        escribirContenido_DRI(raiz.der)
        print(raiz.valor)
        escribirContenido_DRI (raiz.izq)
```

Recorrido D-R-I: primero el hijo derecho, luego la raíz y al final el hijo izquierdo; por eso el contenido sale de mayor a menor.

Retornar un string con la expresión matemática (sin evaluar) de un árbol de expresiones

```
def expresion(raiz):
    assert raiz != nodoVacio
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:
        return str(raiz.valor)
    else:
        strIzquierdo = expresion (raiz.izq)
        strDerecho = expresion (raiz.der)
        operador= raiz.valor
        if operador == "+":
            return "(" + strIzquierdo + "+" + strDerecho + ")"
        elif operador == "-":
            return "(" + strIzquierdo + "-" + strDerecho + ")"
        elif operador == "*":
            return strIzquierdo + "*" + strDerecho
        elif operador == "/":
            return strIzquierdo + "/" + strDerecho
```

Retornar un string con la expresión matemática (sin evaluar) de un árbol de expresiones

```
def expresion(raiz):  
    assert raiz != nodoVacio  
    if raiz.izq == nodoVacio and raiz.der == nodoVacio:  
        return str(raiz.valor)  
    else:  
        strIzquierdo = expresion (raiz.izq)  
        strDerecho = expresion (raiz.der)  
        return "(" + strIzquierdo + raiz.valor + strDerecho + ")"
```

Versión compacta: en vez de un elif por operador, se concatena directamente raiz.valor entre las dos subexpresiones.

Otros propuestos

- ¿Cuántas hojas tiene un árbol binario?
- ¿Cuántos nodos internos tiene un árbol binario?
- Dado dos árboles binarios, ¿Serán iguales?
- Dado un árbol binario, ¿cuál es el menor/mayor valor?
 - Tenga presente que NO es un árbol binario de búsqueda
- Dado un árbol binario, ¿Será un árbol binario de búsqueda?